CSE 3313: Computer Architecture

Sidratul Tanzila Tasmi
Lecturer, Department of Computer Science and Engineering
United International University

Recall: Introduction to MIPS Architecture

- 32 registers in MIPS
  - Registers are numbered from 0 to 31
  - Each register can be referred to by number or name
  - Number references: $0, $1, ... $30, $31
- By convention, each register also has a name to make it easier to code
  - $t0 -$t7 for temporary variables ($8-$15)
  - $ra for return address
- Each MIPS register is 32 bits wide
  - Groups of 32 bits called a word in MIPS

MIPS use Byte Addressable Memory

MIPS uses byte-addressable memory, meaning each memory address points to a single byte.
Since instructions are 4 bytes each, the next instruction must be 4 addresses away from the current one.

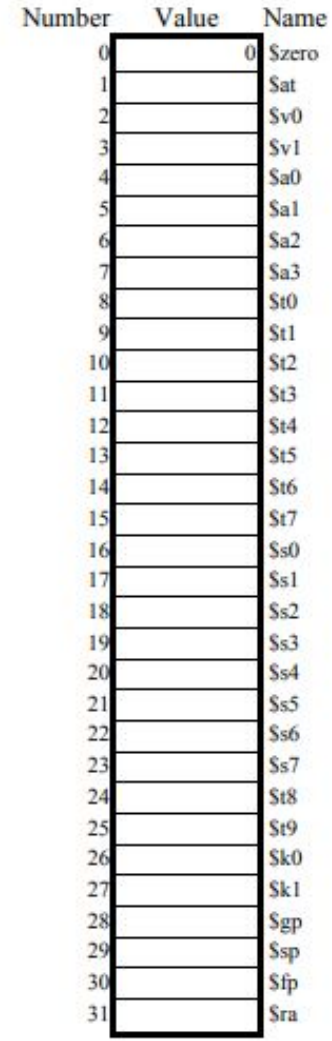
MIPS Register File

Each Register is 32 bit long.

So each register can contain 32 bits.

The 32 bit quantity is called a word.

In the MIPS architecture, memory addresses range from 0 to 4,294,967,295. (2 ^32)

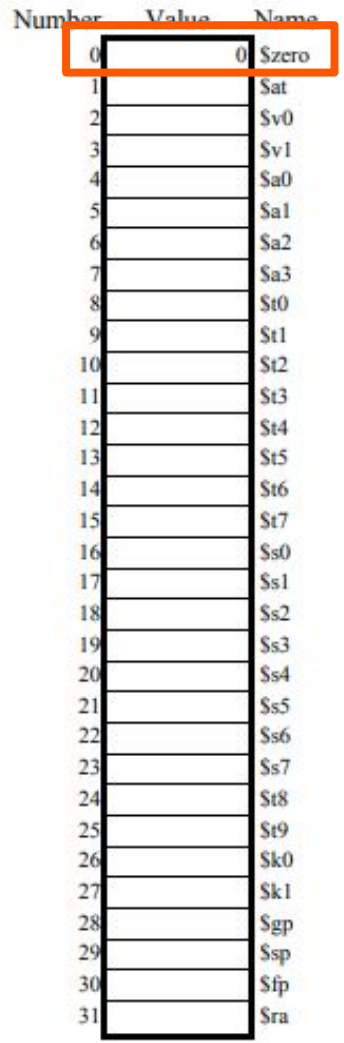


MIPS Register File

The MIPS architecture design includes a special register, $zero ($0), which is hardwired to always hold the value 0. This means:

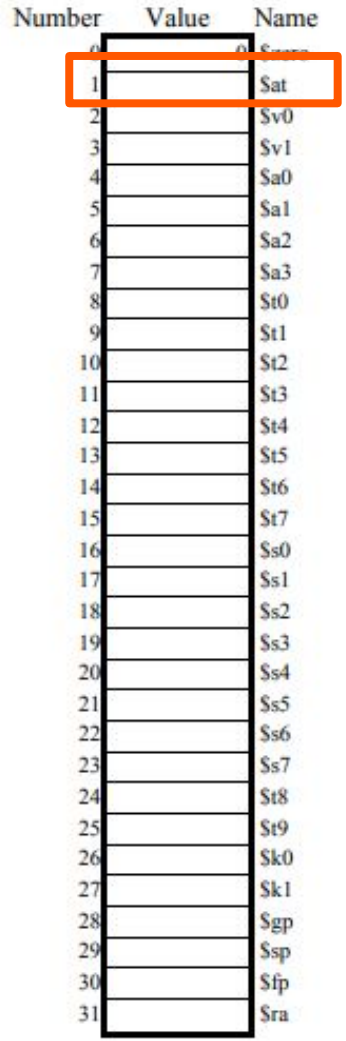
- You cannot modify its value.
- Any attempt to write to $zero will be ignored.
- Reading from $zero always returns 0.

Purpose:
Efficiency, MIPS do not have any Mov instruction!



MIPS Register File

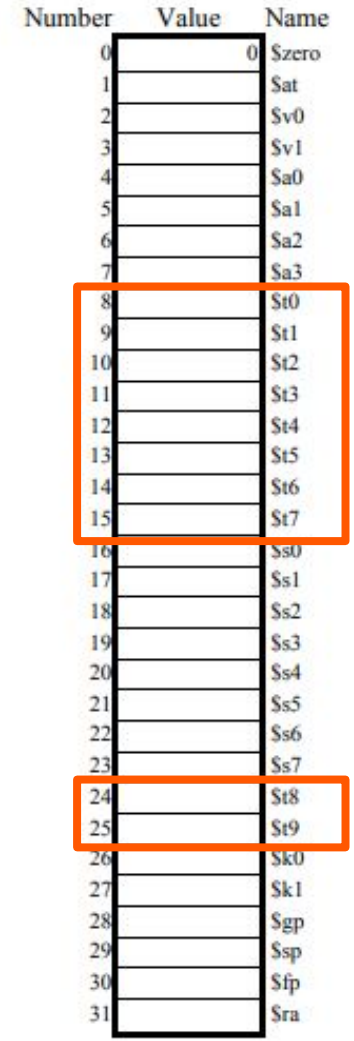
The $at (Assembler Temporary) register is reserved for the assembler and should not be used by programmers. It is used to help execute certain assembly instructions that require extra processing.



MIPS Register File

Temporary Registers:
($t0- $t9)

Used for temporary computations.
Not preserved across function calls
(caller saves if needed).

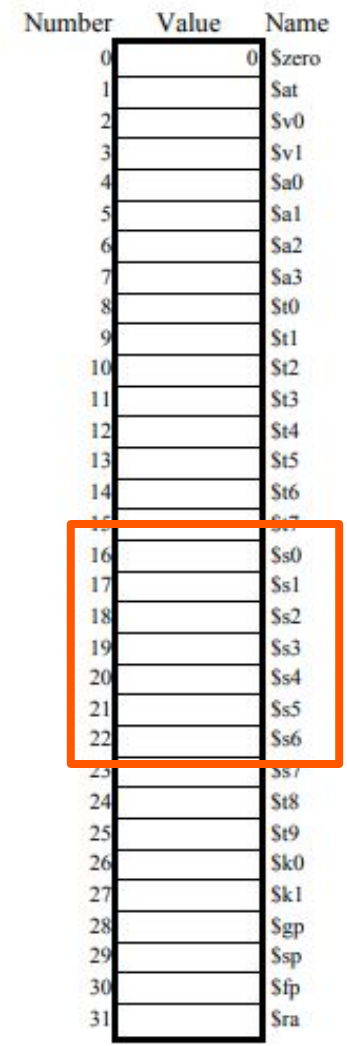


MIPS Register File

Saved Registers:
$s0- $s7)

Used to store important values that must be preserved across function calls.
Callee-saved (function must restore original values before returning).

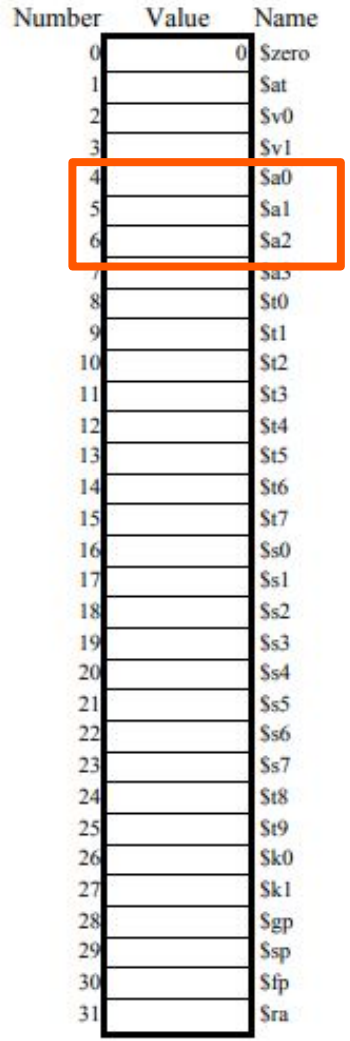
For any variable declaration use this register!



MIPS Register File

Argument Registers ($a0-$a3)

Store function arguments (up to 4). Additional arguments are passed on the stack.



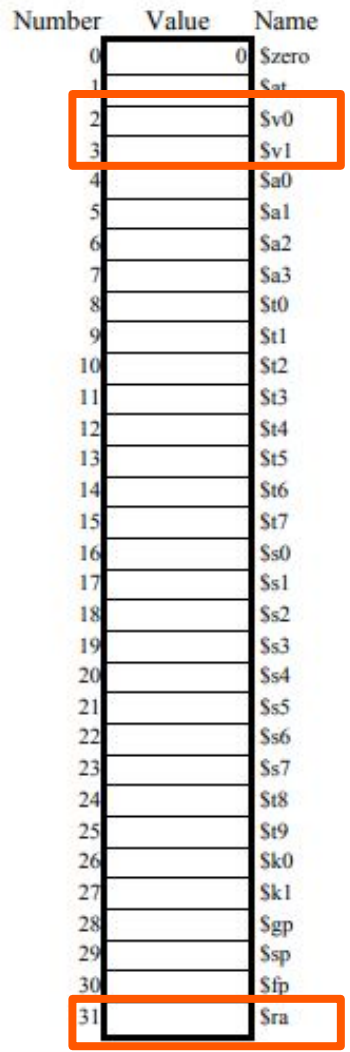
MIPS Register File

Return Value Registers ($v0-$v1)

Store return values from functions

$ra (Return Address):

Stores the return address for function calls..



How Operations Work in MIPS

Type	Destination	Operand 1	Operand 2

How Operations Work in MIPS

Type	Destination	Operand 1	Operand 2
add	c	a	b

C Program:
int c= a+b

Assembly Code:
add c,a,b

How Operations Work in MIPS

Type	Destination	Operand 1	Operand 2
add	c	a	b

C Program:
int c= a+b

Assembly Code:
add c,a,b

Replace variable with saved registers

How Operations Work in MIPS

Type	Destination	Operand 1	Operand 2
add	c	a	b

c= $s0

a= $s1

b= $s2

How Operations Work in MIPS

Type	Destination	Operand 1	Operand 2
add	c	a	b
add	$s0	$s1	$s2

c= $s0

a= $s1

b= $s2

How Operations Work in MIPS

Type	Destination	Operand 1	Operand 2
add	c	a	b
add	$s0	$s1	$s2

Assembly Code:

add $s0, $s1, $s2

Destination Address should not be changed!

Type	Destination	Operand 1	Operand 2
add	a	a	b
add	d	a	c

a= a+b (3+4)
a= 7

d= a+c, c=3
d= 7+3 =10 this is equal to d= a+b+c

Destination Address should not be changed!

a= a+b (3+4)
a= 7

d= a+c, c=3
d= 7+3 =10 this is equal to d= a+b+c

But problem here is if you want to use a somewhere else, the original value of a will be changedm we do not want that

So we use temporary variables to store them, and not destination or saved variables.

Assembly Code

f= (g+h) - (i+j)

f	g	h	i	j
$s0	$s1	$s2	$s3	$s4

Type	Destination	Operand 1	Operand 2
add	$t0	$s1	$s2

Assembly Code

f= (g+h) - (i+j)

f	g	h	i	j
$s0	$s1	$s2	$s3	$s4

Type	Destination	Operand 1	Operand 2
add	$t0	$s1	$s2
add	$t1	$s3	$s4

Assembly Code

f= (g+h) - (i+j)

f	g	h	i	j
$s0	$s1	$s2	$s3	$s4

Type	Destination	Operand 1	Operand 2
add	$t0	$s1	$s2
add	$t1	$s3	$s4
sub	$s0	$t0	$t1

Assembly Code

```
add $t0,$s1,$s2 # register $t0 contains g + h
add $t1,$s3,$s4 # register $t1 contains i + j
sub $s0,$t0,$t1  # f gets $t0 - $t1, which is (g + h)-(i + j)
```

MIPS Reference Sheet

Category	Instruction	Example	Meaning	Comments
Arithmetic	add	`add $s1,$s2,$s3`	`$s1 = $s2 + $s3`	three register operands
	subtract	`sub $s1,$s2,$s3`	`$s1 = $s2 - $s3`	three register operands
Data transfer	load word	`lw $s1,100($s2)`	`$s1 = Memory[$s2 + 100]`	Data from memory to register
	store word	`sw $s1,100($s2)`	`Memory[$s2 + 100] = $s1`	Data from register to memory
Logical	and	`and $s1,$s2,$s3`	`$s1 = $s2 & $s3`	three reg. operands; bit-by-bit AND
	or	`or $s1,$s2,$s3`	`$s1 = $s2 | $s3`	three reg. operands; bit-by-bit OR
	nor	`nor $s1,$s2,$s3`	`$s1 = ~ ($s2 |$s3)`	three reg. operands; bit-by-bit NOR
	and immediate	`andi $s1,$s2,100`	`$s1 = $s2 & 100`	Bit-by-bit AND reg with constant
	or immediate	`ori $s1,$s2,100`	`$s1 = $s2 | 100`	Bit-by-bit OR reg with constant
	shift left logical	`sll $s1,$s2,10`	`$s1 = $s2 << 10`	Shift left by constant
	shift right logical	`srl $$s1,$s2,10`	`$s1 = $s2 >> 10`	Shift right by constant
Conditional branch	branch on equal	`beq $s1,$s2,L`	if (`$s1 == $s2`) go to L	Equal test and branch
	branch on not equal	`bne $s1,$s2,L`	if (`$s1 != $s2`) go to L	Not equal test and branch
	set on less than	`slt $s1,$s2,$s3`	if (`$s2 < $s3`) `$s1 = 1`; else `$s1 = 0`	Compare less than; used with `beq`, `bne`
	set on less than immediate	`slt $s1,$s2,100`	if (`$s2 < 100`) `$s1 = 1`; else `$s1 = 0`	Compare less than immediate; used with `beq`, `bne`
Unconditional jump	jump	`j L`	go to L	Jump to target address
	jump register	`jr $ra`	go to `$ra`	For procedure return
	jump and link	`jal L`	`$ra = PC + 4`; go to L	For procedure call

MIPS Reference Sheet

We covered This!

Category	Instruction	Example	Meaning	Comments
Arithmetic	add	add $s1,$s2,$s3	$s1 = $s2 + $s3	three register operands
	subtract	sub $s1,$s2,$s3	$s1 = $s2 - $s3	three register operands
Data transfer	load word	lw $s1,100($s2)	$s1 = Memory[$s2 + 100]	Data from memory to register
	store word	sw $s1,100($s2)	Memory[$s2 + 100] = $s1	Data from register to memory
Logical	and	and $s1,$s2,$s3	$s1 = $s2 & $s3	three reg. operands; bit-by-bit AND
	or	or $s1,$s2,$s3	$s1 = $s2 | $s3	three reg. operands; bit-by-bit OR
	nor	nor $s1,$s2,$s3	$s1 = ~ ($s2 |$s3)	three reg. operands; bit-by-bit NOR
	and immediate	andi $s1,$s2,100	$s1 = $s2 & 100	Bit-by-bit AND reg with constant
	or immediate	ori $s1,$s2,100	$s1 = $s2 | 100	Bit-by-bit OR reg with constant
	shift left logical	sll $s1,$s2,10	$s1 = $s2 << 10	Shift left by constant
	shift right logical	srl $$s1,$s2,10	$s1 = $s2 >> 10	Shift right by constant
Conditional branch	branch on equal	beq $s1,$s2,L	if ($s1 == $s2) go to L	Equal test and branch
	branch on not equal	bne $s1,$s2,L	if ($s1 != $s2) go to L	Not equal test and branch
	set on less than	slt $s1,$s2,$s3	if ($s2 < $s3) $s1 = 1; else $s1 = 0	Compare less than; used with beq, bne
	set on less than immediate	slt $s1,$s2,100	if ($s2 < 100) $s1 = 1; else $s1 = 0	Compare less than immediate; used with beq, bne
Unconditional jump	jump	j L	go to L	Jump to target address
	jump register	jr $ra	go to $ra	For procedure return
	jump and link	jal L	$ra = PC + 4; go to L	For procedure call

MIPS Reference Sheet

Used for Constant operation

Category	Instruction	Example	Meaning	Comments
Arithmetic	add	add $s1,$s2,$s3	$s1 = $s2 + $s3	three register operands
	subtract	sub $s1,$s2,$s3	$s1 = $s2 - $s3	three register operands
Data transfer	load word	lw $s1,100($s2)	$s1 = Memory[$s2 + 100]	Data from memory to register
	store word	sw $s1,100($s2)	Memory[$s2 + 100] = $s1	Data from register to memory
Logical	and	and $s1,$s2,$s3	$s1 = $s2 & $s3	three reg. operands; bit-by-bit AND
	or	or $s1,$s2,$s3	$s1 = $s2 | $s3	three reg. operands; bit-by-bit OR
	nor	nor $s1,$s2,$s3	$s1 = ~($s2 | $s3)	three reg. operands; bit-by-bit NOR
	and immediate	andi $s1,$s2,100	$s1 = $s2 & 100	Bit-by-bit AND reg with constant
	or immediate	ori $s1,$s2,100	$s1 = $s2 | 100	Bit-by-bit OR reg with constant
	shift left logical	sll $s1,$s2,10	$s1 = $s2 << 10	Shift left by constant
	shift right logical	srl $$s1,$s2,10	$s1 = $s2 >> 10	Shift right by constant
	branch on equal	beq $s1 $s2 L	if ($s1 == $s2) go to L	Equal test and branch
Conditional branch	branch on not equal	bne $s1,$s2,L	if ($s1 != $s2) go to L	Not equal test and branch
	set on less than	slt $s1,$s2,$s3	if ($s2 < $s3) $s1 = 1; else $s1 = 0	Compare less than; used with beq, bne
	set on less than immediate	slt $s1,$s2,100	if ($s2 < 100) $s1 = 1; else $s1 = 0	Compare less than immediate; used with beq, bne
Unconditional jump	jump	j L	go to L	Jump to target address
	jump register	jr $ra	go to $ra	For procedure return
	jump and link	jal L	$ra = PC + 4; go to L	For procedure call

Why other constant operation not present?


UIU
UNITED INTERNATIONAL UNIVERSITY
QUEST
FOR
EXCELLENCE

Thank you!